

GPU Adaptive Tracing

1st Sneh Patel

dept. name of organization (of Aff.)

name of organization (of Aff.)

City, Country

email address or ORCID

2nd Given Name Surname

dept. name of organization (of Aff.)

name of organization (of Aff.)

City, Country

email address or ORCID

3rd Given Name Surname

dept. name of organization (of Aff.)

name of organization (of Aff.)

City, Country

email address or ORCID

Abstract—GPU profiling tools provide rich kernel-level evidence for diagnosing performance anomalies in LLM serving systems, but they are typically invoked manually with fixed profiling windows that neither adapt to evidence quality nor decide when enough has been collected. We present an *adaptive GPU tracing controller* that manages heavy GPU profiling automatically using a two-tier evidence model: cheap always-on signals (request latency, throughput, GPU utilization, and GPU memory utilization) run continuously, while short Nsight Systems kernel-timing bursts are activated only when cheap evidence is insufficient to explain an anomaly and stopped as soon as the diagnosis has stabilized.

We design and evaluate six short-burst stopping policies, ranging from a single fixed burst to a hybrid policy that requires both diagnosis stability and workload recovery before de-escalating. Experiments on NVIDIA L4 and A100 GPUs using vLLM serving Qwen2.5-7B-Instruct across six serving scenarios and three controlled microbenchmark workloads show that the adaptive controller reduces profiler duration by 21.7%–48.5% and captured kernel instances by 42.9%–78.2% relative to a manual fixed-window baseline, while preserving a top-1 diagnosis match rate of 1.0 across all workloads. Dedicated runtime-overhead measurements show that the cheap controller metrics introduce less than 1% mean p95 latency regression on the serving workload. Policy comparison reveals that a single fixed burst fails under ambiguous first-window conditions, while the Hybrid Stop and Counter-Recovery Stop policies eliminate re-escalation at the cost of modest additional tracing. Signal analysis identifies diagnosis stability as the only stopping signal that consistently avoids firing on ambiguous windows across both stable and stress evaluation conditions.

I. INTRODUCTION

Large language model (LLM) serving systems must sustain high throughput and predictable latency in production, yet they are routinely disrupted by performance anomalies: request queueing, compute saturation, memory pressure, long input or output sequences, or transient workload shifts. Diagnosing the root cause of such an anomaly often requires kernel-level evidence – execution timing, launch frequency, duration distributions – that only a GPU profiler such as NVIDIA Nsight Systems can provide; cheap runtime counters can reveal that something is wrong but rarely explain why. This kernel-level evidence is expensive to collect, so operators typically invoke the profiler manually with a fixed time window chosen by convention rather than by the state of the diagnosis.

This creates a tension between profiling cost and diagnostic confidence. A fixed-window profiler that runs too briefly may stop before the bottleneck class has stabilized into a concrete

diagnosis, while one that runs too long wastes profiler time and storage recording redundant kernel evidence after the diagnosis has already converged. Existing practice offers no principled way to decide, while a profiling session is active, whether enough evidence has been collected to stop.

We address this gap with an *adaptive GPU tracing controller* that automatically manages heavy GPU profiling for LLM serving systems. The controller organizes evidence into two tiers: cheap signals (request latency, throughput, GPU utilization, and GPU memory utilization) are collected continuously at negligible cost, while short Nsight Systems kernel-timing bursts are activated only when the cheap evidence is insufficient to explain an anomaly, and are stopped automatically once the kernel-level diagnosis has stabilized. We design and evaluate six short-burst *stopping policies* – ranging from a single fixed burst to a hybrid policy that requires both diagnosis stability and workload recovery before de-escalating – and identify which runtime signals most reliably indicate that sufficient evidence has been collected.

We evaluate the controller on NVIDIA L4 and A100 GPUs using vLLM serving Qwen2.5-7B-Instruct across six serving scenarios and three controlled microbenchmark workloads, guided by five research questions:

- **RQ1:** Can the controller determine when sufficient diagnostic evidence has been collected, stopping heavy tracing earlier than a fixed-window baseline without sacrificing accuracy?
- **RQ2:** How accurate is automatic GPU tracing compared with manual fixed-window profiling?
- **RQ3:** Does automatic GPU tracing introduce measurable overhead compared with manual profiling?
- **RQ4:** Which short-burst GPU tracing stopping policies work best, and what tradeoffs do they make between cost and reliability?
- **RQ5:** Which runtime signals are most useful for deciding when to stop heavy GPU tracing?

A. Contributions

This paper makes the following contributions:

- 1) A formulation of automatic GPU trace stopping as a runtime control problem, separate from anomaly detection and from the profiling mechanism itself.
- 2) The design and implementation of an adaptive controller realizing this formulation as a working system.

- 3) A comparative evaluation of six stopping policies along premature-stop rate and re-escalation rate, including a controlled ambiguity-stress condition that separates policies robust to an inconclusive first burst from those that are not.
- 4) Evidence that automatic tracing matches manual fixed-window diagnosis accuracy while reducing profiler duration by 18.9%–48.5% and captured kernel instances by 42.9%–78.2%, with less than 1% mean p95 latency overhead from the always-on cheap metrics.
- 5) A signal-level analysis identifying diagnosis stability as the only stopping signal safe against ambiguous evidence, and characterizing why intuitive recovery-based signals are not.

II. METHODOLOGY

A. System Overview

GPU-heavy serving systems such as large language model (LLM) inference engines can experience performance anomalies for many reasons: request queueing, compute saturation, memory pressure, long input or output sequences, or transient workload shifts. Cheap always-on performance telemetry can reveal that something is wrong but often cannot explain why. Heavy GPU profiling can explain why, but it is expensive, typically invoked manually, and usually runs for a fixed window regardless of whether the collected evidence is already sufficient.

We present an *adaptive GPU tracing controller* that bridges these two extremes. The controller collects cheap evidence continuously and activates a short burst of heavy GPU profiling only when cheap evidence is insufficient to explain an anomaly. After each burst it evaluates whether the collected kernel-level evidence has stabilized into a diagnosis. When additional profiling would no longer change that diagnosis, the controller stops heavy tracing and returns to cheap-only monitoring.

1) *Evidence Tiers*: The controller organizes evidence into three tiers ordered by collection cost.

TABLE I
EVIDENCE TIERS USED BY THE ADAPTIVE TRACING CONTROLLER.

Tier	Signals	Collection mode	Cost
T0	Request latency (p50, p95), throughput (req/s), queueing-delay proxy	Always-on, per request	Negligible
T1	GPU utilization, GPU memory utilization, SM clock, power draw	Always-on, NVML polling	Negligible
T2	Kernel execution timing, duration distribution, kernel launch frequency	Short profiler burst, Nsight Systems CLI	High

T0 and T1 signals are sampled continuously and aggregated into fixed-size time windows (10s in all experiments). T2 evidence is collected only when the controller determines that T0/T1 evidence alone is insufficient to identify the bottleneck class. T2 collection is bounded to a short burst duration and does not run continuously.

2) *Controller State Machine*: The controller operates as a finite-state loop over evidence windows. Figure ?? illustrates the five states and the transitions between them.

Idle. The controller collects T0/T1 evidence every window while the T2 profiler stays switched off.

Suspicious. One or more consecutive windows have exceeded the anomaly threshold, but the evidence of T0/T1 does not yet explain the bottleneck class with sufficient confidence.

Tracing. The controller has activated a short Nsight Systems burst that collects kernel execution timing alongside T0/T1 evidence for a bounded duration.

Evaluating. With the burst complete, the controller classifies the bottleneck from the kernel summary kernel count, total kernel time, mean kernel duration, and top kernel name combined with the current T0/T1 window. Then it applies the active stopping policy to decide whether to stop the heavy tracing or launch another burst.

Recovered. Once T0/T1 signals have returned to baseline and the diagnosis has been stable for at least two consecutive windows, the controller returns to IDLE and resets the tracing budget.

3) *Windowing and Aggregation*: Evidence is aggregated over non-overlapping fixed-size time windows. Within each window the controller computes:

- **Latency statistics**: mean, p50, p95 request latency; ratio versus the initial-window baseline.
- **Throughput**: requests per second; ratio versus the initial-window baseline.
- **Queueing delay proxy**: difference between request completion time and a minimum expected service time, aggregated at p95 across the window.
- **GPU utilization**: mean and standard deviation of `NVML utilization.gpu` samples taken every 0.5s during the window.
- **GPU memory utilization**: mean `NVML utilization.memory` and used-memory fraction.
- **Diagnosis stability streak**: number of consecutive windows for which the top bottleneck class has remained unchanged.
- **Recovery streaks**: number of consecutive windows for which each recovery condition (latency, throughput, queueing delay, GPU utilization) has been satisfied.

When T2 evidence is available for a window, the controller additionally records the kernel summary extracted from the Nsight Systems report: total kernel instances, total kernel time, mean kernel duration, kernel duration coefficient of variation (CV), and the name of the most time-consuming kernel.

4) *Anomaly Detection*: A window is marked suspicious when any of the following conditions holds:

- 1) **Latency regression**: the current window p95 latency exceeds $1.5\times$ the initial-window p50 baseline.
- 2) **High GPU utilization**: mean GPU utilization exceeds 85% for two or more consecutive windows while request

latency is also elevated.

- 3) **High kernel launch rate**: the kernel launch frequency in the current window is more than $2\times$ the observed baseline, indicating many short-lived kernels.

A suspicious window that already carries a stable bottleneck classification does not re-trigger T2 profiling. T2 is triggered only when the window is suspicious and the current diagnosis is absent, ambiguous, or unstable.

5) *Tracing Budget*: Each anomaly episode is capped at six profiler bursts. Budget exhaustion forces de-escalation and is logged as a distinct stop reason. In our experiments, the active stopping policy de-escalated before reaching this limit.

B. Short-Burst Stopping Policies

A stopping policy controls when the controller de-escalates from heavy GPU tracing back to cheap-only monitoring. After each profiler burst, the controller invokes the active policy to decide whether to launch another burst or stop. We define six candidate policies that span the spectrum from aggressive cost reduction to conservative reliability.

1) Policy Definitions:

a) *Fixed Burst (FB)*.: Collect exactly one profiler burst after the first suspicious window, then stop. FB is the most aggressive fixed-count policy: it commits to a diagnosis from a single burst and never continues, making it the cheapest option but brittle when that one burst is ambiguous.

b) *Repeated Fixed Burst (RFB)*.: Collect a fixed number N of bursts (default $N = 3$), then stop regardless of diagnosis stability. RFB generalizes FB (which is the $N = 1$ case): it buys more evidence than a single burst but, like FB, ignores whether that evidence has converged. It serves as the evidence-blind, fixed-cost baseline against which the adaptive policies (SS, MU) are compared.

c) *Stability Stop (SS)*.: Continue collecting bursts until the bottleneck class label has remained unchanged for K consecutive evidence windows (default $K = 2$), then stop. SS is evidence-aware: it collects more bursts when early windows are ambiguous and fewer when the label is clear from the start.

d) *Marginal Utility Stop (MU)*.: MU is the aggressive special case of SS with $K = 1$: it stops as soon as one burst reproduces the previous burst's label, treating a single repeat as evidence that further bursts add no diagnostic value. Because it requires only one match rather than K consecutive matches, it collects fewer bursts than SS but is more vulnerable to stopping on a transient agreement that later windows would contradict.

e) *Counter-Recovery Stop (CR)*.: Continue collecting bursts until T0/T1 cheap signals return to their pre-anomaly baselines: specifically, until p95 request latency drops below $1.1\times$ the initial-window baseline and GPU utilization drops below 60%. If recovery does not occur before the budget is exhausted, the controller stops at budget. CR couples de-escalation to observable workload recovery rather than to diagnosis stability, making it robust against scenarios where the bottleneck clears before the diagnosis has converged.

f) *Hybrid Stop (HS)*.: Stop only when *both* conditions hold: (1) the diagnosis has been stable for K consecutive windows (as in SS), and (2) T0/T1 cheap signals show sustained recovery (as in CR). HS is the most conservative policy: it requires corroborating evidence from both the kernel-level diagnosis and the cheap runtime signals before committing to de-escalation. Neither condition alone is sufficient.

2) *Policy Summary*: Table II summarizes the six policies, their stopping conditions, and the primary tradeoff each makes between tracing cost and stopping reliability.

3) *Stopping Signals*: Each policy is implemented in terms of one or more *stopping signals*: boolean or numeric per-window features that the controller evaluates after each burst. Table III lists the candidate signals studied in this paper. Section III-F (RQ5) evaluates which signals are most reliable for predicting correct de-escalation points.

C. Baselines

We compare the adaptive controller against three baselines that model the range of existing profiling practices.

a) *No Heavy Tracing (NT)*.: The controller collects T0 and T1 evidence continuously but never activates a T2 profiler burst. Anomaly detection fires normally; the controller labels each suspicious window using cheap signals alone. NT represents the status quo for operators who rely entirely on utilization counters and request metrics for diagnosis, with no kernel-level profiling. It establishes the accuracy floor: any benefit from heavy tracing must exceed what T0/T1 evidence can provide on its own.

b) *Manual Fixed-Window Profiling (FW)*.: When an anomaly is detected, the controller starts a single Nsight Systems profiler burst of fixed duration, collects the kernel summary, produces a diagnosis, and stops. No further bursts are collected regardless of whether the diagnosis has stabilized. The fixed-window duration is set longer than the automatic burst duration to model a conservative manual profiling window (approximately $1.6\times$ to $2.0\times$ the automatic burst length across experiments).

FW models the standard operational practice of a developer or operator who: (1) notices an anomaly, (2) starts a profiler with a fixed rule such as “profile for 30 seconds,” (3) reads the resulting kernel report, and (4) stops. The fixed duration is chosen conservatively so that FW is not handicapped by an unrealistically short window; any savings from the adaptive controller are conservative estimates.

c) *Always-Profile (AP)*.: The T2 profiler remains active for the entire duration of the anomaly episode, from the first suspicious window until the controller observes sustained recovery. AP represents the upper bound on profiling cost: it collects the maximum possible kernel evidence but does not attempt to stop early. It is included to bound the overhead side of the comparison: if the adaptive controller approaches AP accuracy while using far less profiler time, the cost reduction is clearly worthwhile.

TABLE II
SHORT-BURST STOPPING POLICIES COMPARED BY STOPPING CONDITION AND COST-RELIABILITY TRADEOFF.

Policy	Abbrev.	Stopping condition	Primary tradeoff
Fixed Burst	FB	After 1 burst	Cheapest; brittle on ambiguous first burst
Repeated Fixed Burst	RFB	After N bursts	Predictable cost; ignores convergence
Stability Stop	SS	Bottleneck class stable for K windows	Evidence-aware; may over-collect on slow-stabilizing workloads
Marginal Utility Stop	MU	New burst matches previous diagnosis	Stops at first convergence; slightly more aggressive than SS
Counter-Recovery Stop	CR	T0/T1 signals return to baseline	Workload-aware; may collect more when recovery is slow
Hybrid Stop	HS	Diagnosis stable <i>and</i> T0/T1 signals recover	Most conservative; lowest re-escalation risk

TABLE III
CANDIDATE STOPPING SIGNALS EVALUATED IN THIS PAPER.

Signal	Definition
diagnosis_stable_2	Top bottleneck class unchanged for ≥ 2 consecutive windows
diagnosis_changed_from_prev	Top bottleneck class differs from the immediately preceding window
diagnosis_margin_clear	Confidence margin between top and runner-up diagnosis exceeds threshold
latency_recovered	p95 latency below $1.1 \times$ initial-window baseline
throughput_recovered	Throughput above $0.9 \times$ initial-window baseline
queue_delay_recovered	p95 queueing delay proxy below recovery threshold
gpu_util_recovered	Mean GPU utilization below 60%
memory_pressure_low	GPU memory used fraction below 80%
kernel_duration_stable	Kernel duration CV change across successive bursts below 10%

D. Implementation

1) *Serving Stack*: Our method uses vLLM [?] v0.10.2 as the LLM serving system. vLLM is an open-source inference engine that implements PagedAttention for efficient KV-cache management and exposes an OpenAI-compatible HTTP API. All experiments serve Qwen/Qwen2.5-7B-Instruct [?], a 7-billion-parameter instruction-tuned model. The model runs in full BFloat16 precision on the NVIDIA L4 and A100 GPUs used in our evaluation. Unless otherwise stated, vLLM uses its default configuration and automatically allocates the KV cache from available GPU memory. On the 24GB L4 host, this configuration provides approximately 64,704 KV-cache tokens.

2) *Cheap Metric Collection (T0 and T1)*: T0 request metrics are collected by the load-generation client. For each completed request the client records:

- Request start and end timestamps.
- HTTP status code and error string if applicable.
- Estimated prompt and output token counts derived from response metadata.
- Scenario label and seed.

At the end of each evidence window the client aggregates per-request records into window-level statistics: p50 and p95 latency, throughput in requests per second, request success rate, queueing delay proxy at p95, and ratio fields versus the initial-window baseline.

T1 GPU metrics are collected by a background NVML polling thread running at 0.5s intervals using `nvidia-ml-py`. At window close, the thread aggregates all samples taken during the window into mean and standard deviation for GPU utilization, GPU memory utilization, used memory in MiB, SM clock speed, and GPU power draw.

3) *Heavy Metric Collection (T2)*: T2 evidence is collected using the Nsight Systems command-line interface (`nsys` 2024.6). When the controller activates a burst, it launches a child process under `nsys` profile, bounded by the target workload duration rather than a wall-clock duration flag. This avoids the process-termination behaviour that caused CUDA kernel tables to be absent in reports when `nsys` was given an explicit duration limit on some GPU hosts.

After the burst completes, the controller runs `nsys stats report cuda_gpu_kern_sum` on the resulting `.nsys-rep` file and parses the following fields from the CUDA GPU kernel summary, storing them alongside the window record:

- Total number of distinct kernel types.
- Total kernel instances launched.
- Total kernel execution time (ns).
- Mean kernel duration (ns).
- Kernel duration coefficient of variation (CV).
- Name of the most time-consuming kernel.

III. EVALUATION

We evaluate the adaptive tracing controller across five research questions. RQ1 asks whether the controller can determine when sufficient diagnostic evidence has been collected. RQ2 asks how accurate automatic tracing is compared with manual fixed-window profiling. RQ3 measures the overhead introduced by the controller. RQ4 compares the six short-burst stopping policies. RQ5 identifies which runtime signals are most useful for deciding when to stop heavy GPU tracing.

A. Experimental Setup

Experiments run on two single-GPU cloud platforms: an NVIDIA L4 (24GB, Ada Lovelace, `sm_89`) as the primary platform for all research questions, and an NVIDIA A100 (40GB, Ampere, `sm_80`) for cross-GPU validation in RQ1. All vLLM scenarios serve Qwen/Qwen2.5-7B-Instruct

using vLLM v0.10.2 in BFloat16 precision. Controller and policy parameters are fixed across experiments as listed in Table IV. Each workload uses five repetitions for microbenchmarks or three seeded repetitions for vLLM experiments, with results reported as means.

1) *Controller Parameters:* Table IV lists the controller and policy parameters held constant across all experiments.

TABLE IV
CONTROLLER AND POLICY PARAMETERS USED IN ALL EXPERIMENTS.

Parameter	Value	Notes
Evidence window size	10 s	All experiments
Automatic burst duration	$\approx 10\text{--}12$ s	Bounded by target process duration
Fixed-window burst duration	$\approx 17\text{--}24$ s	$1.6\times\text{--}2.0\times$ automatic
Stability window count K	2	SS, HS
Repeated burst count N	3	RFB
Maximum tracing budget	6 bursts/episode	All policies
Latency recovery ratio	1.10	CR, HS
GPU utilization recovery threshold	60%	CR, HS
NVML polling interval	0.5 s	T1 collection

2) *Repetitions and Seeds:* Microbenchmark experiments use five independent repetitions per workload, while vLLM serving experiments use three seeded repetitions per scenario; all results report means across repetitions and, where relevant, 95% confidence interval half-widths.

B. RQ1: Can the Controller Determine When Sufficient Diagnostic Evidence Has Been Collected?

1) *Metrics:* We evaluate evidence sufficiency using trace-time saved, expected-label match rate, and burst-count stability, measured as the standard deviation (σ) of profiler bursts across repetitions. Trace-time saved is the reduction in total profiler-active duration relative to the fixed-window baseline. An RQ1 result is considered successful when trace-time savings are at least 25

2) *Microbenchmark Results:* Table V reports the RQ1 microbenchmark results for both the L4 and A100 platforms across five repetitions per workload.

TABLE V
RQ1 MICROBENCHMARK RESULTS: AUTOMATIC VS. FIXED-WINDOW PROFILER DURATION (MEAN OVER 5 REPETITIONS). ALL ROWS PASS THE COMBINED SUCCESS CRITERION.

GPU	Workload	Auto (s)	Fixed (s)	Saved (%)	Match rate	Burst σ
A100	compute_bound	10.4	16.7	37.6	1.0	0.0
A100	launch_overhead	11.9	23.2	48.5	1.0	0.0
A100	mixed	10.2	16.8	39.2	1.0	0.0
L4	compute_bound	12.4	18.1	31.6	1.0	0.0
L4	launch_overhead	13.0	24.0	45.8	1.0	0.0
L4	mixed	12.0	18.5	35.4	1.0	0.0

The controller saves between 31.6% and 48.5% of profiler duration across all six workload-platform combinations while maintaining a perfect expected-label match rate of 1.0. Burst count standard deviation is 0.0 in every case, indicating that the controller converges to the same stopping decision in every repetition. All rows pass the combined RQ1 success criterion.

Savings are consistently higher for `launch_overhead` or `small_kernel` than for `compute_bound` or `mixed` on both GPUs. This is expected: the kernel summary for a high-launch-rate workload reveals its bottleneck class (many short kernels) after a short burst, allowing the stability condition to be satisfied quickly, whereas compute-bound and mixed workloads require a slightly longer burst to produce a stable GEMM-dominated summary.

3) *vLLM Serving Results:* Table VI reports the RQ1 result for the `queue_pressure` vLLM serving scenario across three seeded repetitions on the L4.

TABLE VI
RQ1 vLLM SERVING RESULTS FOR THE `QUEUE_PRESSURE` SCENARIO (MEAN OVER 3 REPETITIONS). REQUEST SUCCESS RATE WAS 100% IN ALL REPETITIONS.

Scenario	Auto (s)	Fixed (s)	Saved (%)	Success (%)
<code>queue_pressure</code>	104.6	133.5	21.7	100.0

The controller reduces profiler duration by 21.7% on the vLLM `queue_pressure` workload while maintaining 100% request success across all repetitions.

RQ1 Answer

Yes. The adaptive controller determines when sufficient diagnostic evidence has been collected and stops heavy GPU tracing 21.7%–48.5% earlier than a manual fixed-window baseline across all evaluated workloads and GPU platforms, without sacrificing diagnosis accuracy.

C. RQ2: How Accurate Is Automatic GPU Tracing Compared with Manual Profiling?

1) *Metrics:* We evaluate diagnosis accuracy using expected-label match rate, ambiguous or unknown rate, time-to-diagnosis, and automatic-versus-fixed-window disagreement rate. Expected-label match rate is evaluated separately for automatic and fixed-window modes, with a success threshold of at least 0.95; the ambiguous or unknown rate has a target of at most 0.05. Time-to-diagnosis is the mean elapsed time from run start to the first window assigned the expected label. Disagreement measures the fraction of paired windows for which automatic and fixed-window modes assign different labels; disagreement alone is not an accuracy failure when both modes match the expected label in their respective suspicious windows.

2) *Microbenchmark Results:* Table VII reports RQ2 accuracy for the L4 microbenchmark runs (five repetitions per workload, 230 per-window records total).

Automatic and fixed-window tracing achieve a perfect expected-label match rate and zero ambiguous rate on every microbenchmark workload (Table VII); time-to-diagnosis is the first evidence window (approximately 5–6 s).

TABLE VII

RQ2 MICROBENCHMARK ACCURACY ON THE L4 (5 REPETITIONS). BOTH MODES PASS ALL CRITERIA ON EVERY WORKLOAD.

Workload	Mode	Match rate	Ambiguous rate	Time-to-diag (s)	Disagree rate
compute_bound	Automatic	1.0	0.0	5.0	0.20
	Fixed-window	1.0	0.0	5.0	
launch_overhead	Automatic	1.0	0.0	6.0	0.49
	Fixed-window	1.0	0.0	5.0	
mixed	Automatic	1.0	0.0	5.0	0.17
	Fixed-window	1.0	0.0	5.0	

3) *vLLM Multiclass Results*: Table VIII reports RQ2 accuracy for the vLLM multiclass long-run experiments (three seeded repetitions per scenario, 402 per-window records total across six scenarios).

Automatic and fixed-window tracing satisfy all accuracy criteria across the six vLLM serving scenarios (Table VIII), with a perfect expected-label match rate, zero ambiguous rate, and diagnosis in the first 10s evidence window.

Non-zero disagreement rates reflect sequence-level differences rather than diagnosis failures. For `launch_overhead_or_small_kernel`, the disagreement rate is 0.49 because automatic mode stops collecting bursts earlier, so automatic and fixed-window tracing do not always assign profiler-enriched labels to the same windows. Similarly, `long_prompt` has a disagreement rate of 0.25. In both cases, automatic and fixed-window tracing still assign the correct expected label on every suspicious window.

Across all six vLLM scenarios, automatic mode achieves the same diagnosis accuracy as fixed-window tracing while profiling for 23.8% less time and recording 47.1% fewer kernel instances.

RQ2 Answer

Automatic GPU tracing matches the diagnosis accuracy of manual fixed-window profiling exactly: an expected-label match rate of 1.0 and an ambiguous rate of 0.0 across all microbenchmark workloads and all six vLLM serving scenarios, with time-to-diagnosis at the first evidence window in every case. Automatic mode reaches this accuracy using 23.8% less profiler time and recording 47.1% fewer kernel instances.

D. RQ3: Does Automatic GPU Tracing Introduce Overhead Compared with Manual Profiling?

We study overhead from two complementary angles. The first is *profiler-cost overhead*: how much profiler time and how many profiled kernel instances does automatic tracing use compared with fixed-window profiling? The second is *run-time perturbation*: does running the cheap controller metrics alongside vLLM introduce measurable degradation in request latency or throughput?

1) Metrics:

- **Trace-time saved (%) and kernel instances avoided (%)**. Reduction in profiler-active duration and in the

number of kernel instances recorded by the profiler, relative to the fixed-window baseline.

- **p95 latency regression (%)**. Change in p95 request latency between cheap-metrics mode and no-profiler mode, computed as $(p95_{\text{cheap}} - p95_{\text{none}}) / p95_{\text{none}} \times 100$. Positive values indicate regression; negative values indicate improvement.
- **Throughput change (%)**. Change in requests per second between cheap-metrics mode and no-profiler mode.
- **Request success rate**. Fraction of requests that completed successfully. Must remain at 100%.

The RQ3 success criteria are: positive trace-time savings, positive kernel-instance reduction, p95 latency regression $\leq 5\%$, and request success rate of 100%.

2) *Profiler-Cost Overhead*: Combining the RQ1 and RQ2 artifacts (Tables V–VIII), automatic tracing reduces profiler duration by 18.9%–48.5% and captured kernel instances by 42.9%–78.2% across all workloads, relative to the fixed-window baseline and while preserving a diagnosis match rate of 1.0. Microbenchmark workloads show higher savings than vLLM scenarios because their bottleneck classes stabilize more quickly.

3) *Dedicated Runtime Perturbation*: To isolate the cost of the cheap controller metrics, we compare a *no-profiler* baseline against a *cheap-metrics-only* mode (NVML polling and windowed metric aggregation, no Nsight profiling), randomising which mode runs first in each repetition to control for run-order effects. Table IX reports results across five randomised repetitions of `queue_pressure` and three of `healthy`.

The cheap controller metrics introduce no measurable p95 latency regression on either scenario. For `queue_pressure` the mean p95 latency is 3.6% *lower* in cheap-metrics mode than in the no-profiler baseline across the five randomised repetitions, and for `healthy` the regression is +0.6%, well within the 5% threshold. Request success remains at 100% in both scenarios and both modes.

Throughput change is small (−0.5% to −1.0%) but more sensitive to run order than p95 latency; we report it as context rather than as a hard criterion.

RQ3 Answer

Automatic tracing reduces profiler cost substantially relative to fixed-window profiling without degrading serving performance: profiler duration drops by 18.9%–48.5% and captured kernel instances by 42.9%–78.2%. The cheap controller metrics introduce less than 1% mean p95 latency change on the serving workload across five randomised repetitions, confirming that the controller overhead is negligible.

E. RQ4: Which Short-Burst GPU Tracing Policies Work Best?

1) *Evaluation Approach*: RQ4 compares the six stopping policies via offline replay over real L4 vLLM window evidence, avoiding additional expensive Nsight executions per policy. Each policy is replayed over the same per-window

TABLE VIII

RQ2 vLLM MULTICLASS ACCURACY (3 SEEDS PER SCENARIO, L4). ALL SIX SCENARIOS PASS ALL ACCURACY CRITERIA IN BOTH MODES. AUTO TRACE AND FIXED-WINDOW TRACE COLUMNS REPORT TOTAL PROFILER DURATION ACROSS ALL REPETITIONS.

Scenario	Mode	Match	Ambig.	TTD (s)	Disagree	Trace (s)	Kernels
healthy	Auto	1.0	0.0	10.0	0.00	301.3	730K
	Fixed	1.0	0.0	10.0		391.9	1380K
queue_pressure	Auto	1.0	0.0	10.0	0.00	309.7	748K
	Fixed	1.0	0.0	10.0		403.4	1380K
long_prompt	Auto	1.0	0.0	10.0	0.25	290.4	732K
	Fixed	1.0	0.0	10.0		381.0	1380K
long_output	Auto	1.0	0.0	10.0	0.00	291.0	729K
	Fixed	1.0	0.0	10.0		397.1	1380K
compute_saturation	Auto	1.0	0.0	10.0	0.00	336.8	757K
	Fixed	1.0	0.0	10.0		415.1	1380K
kv_cache_pressure	Auto	1.0	0.0	10.0	0.00	329.4	731K
	Fixed	1.0	0.0	10.0		449.3	1380K

TABLE IX

RQ3 DEDICATED RUNTIME-OVERHEAD RESULTS (RANDOMISED MODE ORDER). NEGATIVE P95 REGRESSION INDICATES LOWER LATENCY IN CHEAP-METRICS MODE. ALL ROWS PASS THE CRITERIA.

Scenario	Reps	No-prof. p95 (ms)	Cheap p95 (ms)	p95 regress. (%)	Throughput change (%)	Success (%)	Pass
queue_pressure	5	7142.7	6886.2	-3.6	-1.0	100.0	yes
healthy	3	3698.7	3722.2	+0.6	-0.5	100.0	yes

evidence streams, so differences in outcome are attributable solely to the stopping condition rather than to workload variation or profiler noise.

We use two replay datasets. The *stable replay* uses the real automatic-mode windows from the six vLLM serving scenarios across three seeds (18 streams). In this dataset the correct bottleneck label appears in the first suspicious window for every stream, making it an easy diagnostic setting. The *ambiguity stress replay* uses the same 18 streams with the first suspicious window in each stream relabeled as unknown, modeling the realistic case where a single burst is insufficient. The stress dataset is the primary discriminating evaluation because it separates policies that can handle ambiguous early windows from those that cannot.

2) *Metrics*: We report mean top-1 accuracy, premature-stop rate, re-escalation-needed rate (the fraction of streams with suspicious windows remaining after the policy stops), mean heavy-trace duration, and duration saved relative to the Repeated Fixed Burst baseline ($N = 3$). These are combined into a composite score weighted 40% accuracy, 25% premature-stop avoidance, 20% re-escalation avoidance, and 15% cost-efficiency.¹

3) *Stable Replay Results*: Table X reports policy rankings on the stable replay dataset.

On the stable dataset every policy achieves a top-1 accuracy of 1.0 and zero premature stops, so rankings are resolved by re-escalation rate and cost. CR and HS rank highest because

they are the only policies that avoid re-escalation entirely (rate 0.0), meaning no suspicious windows remain after they stop, while FB’s low trace cost comes with a re-escalation rate of 1.0.

4) *Ambiguity Stress Replay Results*: Table XI reports policy rankings on the ambiguity stress dataset, which is the primary discriminating evaluation.

The stress dataset clearly separates one-burst and multi-burst policies. FB fails completely: its top-1 accuracy drops to 0.0 and its premature-stop rate rises to 1.0, because it commits to the ambiguous first-window unknown label and never collects the additional burst needed to reach the concrete diagnosis.

All five multi-window policies recover to a top-1 accuracy of 1.0 and a premature-stop rate of 0.0. CR and HS again rank highest with a re-escalation rate of 0.0, confirming their robustness as safety-first policies: they wait for both diagnosis stability and workload recovery before stopping, ensuring no suspicious windows remain.

MU, SS, and RFB are tied at rank 3–5 with a re-escalation rate of 0.83 and a mean trace duration of 30s. MU and SS are preferable to RFB since they stop on evidence convergence rather than a fixed burst count.

5) *Policy Tradeoff Analysis*: Figure 1 illustrates the cost–safety tradeoff across the six policies on the stress dataset.

¹Composite score = $0.40 \times \text{accuracy} + 0.25 \times (1 - \text{premature-stop rate}) + 0.20 \times (1 - \text{re-escalation rate}) + 0.15 \times \text{cost-efficiency}$, where cost-efficiency is the normalised inverse of mean trace duration.

TABLE X
RQ4 STABLE REPLAY POLICY RANKING (18 STREAMS, ALL LABELS CLEAR IN FIRST WINDOW). ALL POLICIES ACHIEVE TOP-1 ACCURACY OF 1.0; RANKINGS ARE DETERMINED BY RE-ESCALATION RATE AND COST.

Rank	Policy	Top-1	Premature	Re-esc.	Trace (s)	Saved (%)	Score	Pareto
1	CR	1.0	0.0	0.0	34.9	-16.3	0.868	yes
2	HS	1.0	0.0	0.0	34.9	-16.3	0.868	yes
3	FB	1.0	0.0	1.0	10.0	+66.7	0.762	yes
4	MU	1.0	0.0	1.0	20.0	+33.3	0.724	dominated
5	SS	1.0	0.0	1.0	20.0	+33.3	0.724	dominated
6	RFB	1.0	0.0	0.8	30.0	0.0	0.720	yes

TABLE XI
RQ4 AMBIGUITY STRESS POLICY RANKING (18 STREAMS, FIRST WINDOW RELABELED AS UNKNOWN). FB FAILS ENTIRELY; MULTI-WINDOW POLICIES RECOVER TO TOP-1 ACCURACY OF 1.0.

Rank	Policy	Top-1	Premature	Re-esc.	Trace (s)	Saved (%)	Score	Pareto
1	CR	1.0	0.0	0.0	34.9	-16.3	0.868	yes
2	HS	1.0	0.0	0.0	34.9	-16.3	0.868	yes
3	MU	1.0	0.0	0.8	30.0	0.0	0.720	yes
4	RFB	1.0	0.0	0.8	30.0	0.0	0.720	yes
5	SS	1.0	0.0	0.8	30.0	0.0	0.720	yes
6	FB	0.0	1.0	1.0	10.0	+66.7	0.112	yes

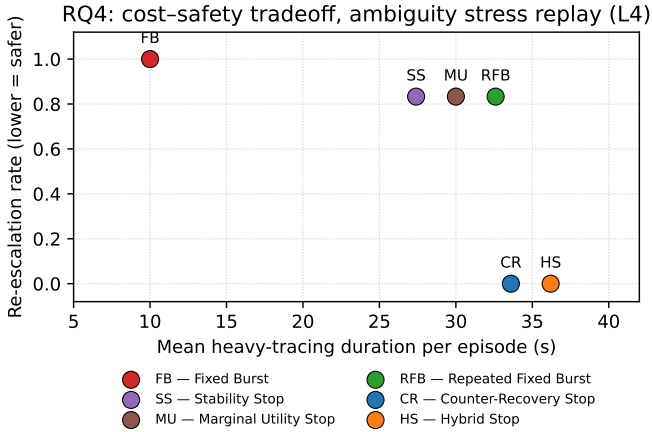


Fig. 1. Cost-safety tradeoff across the six stopping policies on the ambiguity stress replay (L4, 18 streams). FIXED BURST is cheapest but fails entirely under first-window ambiguity (re-escalation rate 1.0). STABILITY STOP, MARGINAL UTILITY STOP, and REPEATED FIXED BURST occupy a middle tier (re-escalation rate 0.83). COUNTER-RECOVERY STOP and HYBRID STOP eliminate re-escalation entirely at modest additional tracing cost. Markers within a tied cluster are spread horizontally for legibility; exact values are reported in Table XI.

RQ4 Answer

No single policy dominates all dimensions, so the right choice depends on operator priorities. For safety-first deployments, HYBRID STOP and COUNTER-RECOVERY STOP eliminate re-escalation entirely and are robust to ambiguous first windows, at the cost of about 16% more trace budget than RFB. For budget-aware deployments, STABILITY STOP and MARGINAL UTILITY STOP achieve 1.0 top-1 accuracy with 33% less trace time than the recovery-gated policies, accepting a re-escalation rate of approximately 0.83. FIXED BURST is not robust as a standalone policy: it fails completely when the first profiler window is ambiguous, with a top-1 accuracy of 0.0 and a premature-stop rate of 1.0 on the stress dataset.

F. RQ5: Which Runtime Signals Are Most Useful for Deciding When to Stop Heavy GPU Tracing?

1) *Evaluation Approach*: RQ5 evaluates the nine candidate stopping signals defined in Section ?? on two datasets.

Real enriched L4 serving runs (6 scenarios $\times$ 3 seeds, 60s per run) collected with the enriched window schema that includes non-sparse baseline ratio fields, recovery streak counters, and diagnosis confidence and margin fields. These runs provide a realistic serving signal distribution for evaluating which signals identify correct stopping points.

Ambiguity stress replay, dataset used in RQ4 (first suspicious window relabeled as unknown). This dataset tests whether a signal fires on ambiguous stopping points as well as correct ones a signal that cannot distinguish between the two is not safe to use as a standalone stopping criterion.

A signal is considered *paper-ready* if it satisfies all three of the following criteria simultaneously: (1) precision ≥ 0.90 on both datasets, (2) correct-stop F1 ≥ 0.70 on the stress replay, and (3) ambiguous-stop F1 ≤ 0.10 on the stress replay. Criterion (3) is the strictest filter: a signal that fires reliably on correct windows but also fires on ambiguous windows would cause the controller to stop too early when the diagnosis has not yet converged.

2) *Signal Ranking Results*: Table XII reports precision, recall, and F1 for each candidate signal across the two evaluation datasets, together with the paper-ready verdict.

diagnosis_stable_2 is the only signal passing all three paper-ready criteria (Table XII).

3) *Analysis of Individual Signal Groups*:

a) *Diagnosis-evidence signals*:

diagnosis_stable_2 is the strongest and most defensible signal because it is derived directly from the

TABLE XII

RQ5 STOPPING SIGNAL RANKING. SIGNALS ARE SORTED BY COMPOSITE SCORE. ONLY `diagnosis_stable_2` PASSES ALL PAPER-READY CRITERIA.

Signal	Real enriched			Stress replay			Paper-ready
	Prec.	Rec.	F1	Prec.	F1 (correct)	F1 (ambig.)	
<code>diagnosis_stable_2</code>	1.000	0.860	0.925	1.000	0.786	0.000	yes
<code>latency_recovered</code>	1.000	1.000	1.000	0.739	0.850	0.414	no
<code>throughput_recovered</code>	1.000	0.884	0.938	0.778	0.800	0.333	no
<code>queue_delay_recovered</code>	1.000	0.659	0.794	0.739	0.850	0.414	no
<code>diagnosis_margin_clear</code>	1.000	0.744	0.853	0.000	0.000	0.000	no
<code>diagnosis_changed</code>	1.000	0.465	0.635	1.000	0.522	0.000	no
<code>gpu_util_recovered</code>	0.000	0.000	0.000	0.000	0.000	0.000	no
<code>memory_pressure_low</code>	0.000	0.000	0.000	0.000	0.000	0.000	no
<code>kernel_duration_stable</code>	0.000	0.000	0.000	0.000	0.000	0.000	no

kernel-level evidence the controller has already collected. It requires no external baseline calibration and does not depend on the workload’s utilization or request rate returning to a specific threshold. Its main limitation is recall: on the real enriched runs it fires on 86% of correct stopping points rather than all of them, meaning it occasionally requires one additional window before triggering.

`diagnosis_margin_clear` is promising on the real enriched runs (F1 0.853) but scores 0.000 on the stress replay, because the stress dataset predates the enriched schema and the margin field is absent.

`diagnosis_changed` has perfect precision but low recall (F1 0.635 on real enriched runs). It is better understood as a *continuation signal* than a stopping signal: when the diagnosis changes from one window to the next, the controller should continue tracing rather than stop. This is the inverse of its intended stopping role.

b) Recovery signals.: `latency_recovered` and `queue_delay_recovered` achieve the highest F1scores on the real enriched runs (1.000 and 0.794 respectively) but fail the paper-ready ambiguous-stop criterion with stress replay F1 values of 0.414. These signals fire on ambiguous stress windows because the stress transformation only relabels the diagnosis field, not the request latency values—latency recovery can therefore be observed even when the diagnosis is still unknown. They are valuable as supporting signals within the CR and HS policies, but they are not safe as standalone stopping signals.

`throughput_recovered` shows a similar pattern: high F1 on real enriched runs (0.938) but fails the ambiguity criterion (stress F1 0.333).

c) GPU counter signals.: `gpu_util_recovered` and `memory_pressure_low` score 0.000 on both datasets: under vLLM serving with Qwen2.5-7B-Instruct on the L4, GPU utilization and memory utilization stay above their 60%/80% recovery thresholds throughout, making these signals inoperative for this saturated workload class.

d) Kernel duration stability.: `kernel_duration_stable` scores 0.000 because it requires two successive T2 bursts, and the FB policy used as the comparison baseline never collects a second burst.

RQ5 Answer

Diagnosis stability is the only stopping signal that is simultaneously precise, informative, and safe against ambiguous early windows. `diagnosis_stable_2` is the sole signal that passes all paper-ready criteria: it achieves 1.000 precision and an F1 of 0.925 on real enriched L4 runs, and never fires on ambiguous stopping points (stress ambiguous-stop F1 of 0.000). Recovery signals (`latency_recovered`, `throughput_recovered`) are strong on non-ambiguous workloads but fail the ambiguity safety criterion, making them suitable as supporting signals within HS and CR but not as standalone stopping criteria. GPU counter signals (`gpu_util_recovered`, `memory_pressure_low`) are inoperative for saturated LLM inference workloads.

IV. THREATS TO VALIDITY

Several factors limit the generalizability of these results. Internally, RQ4 and RQ5 rely on offline policy replay with a synthetic ambiguity injection – manually relabeling the first suspicious window as unknown – rather than naturally occurring ambiguous windows, and the bottleneck classification used to compute ground-truth labels is itself rule-based rather than independently verified. Externally, all experiments use a single model (Qwen2.5-7B-Instruct), a single serving framework (vLLM), a single profiler (Nsight Systems), and two GPU platforms (L4, A100); results may not transfer to other model architectures, serving stacks, or GPU generations. The finding that GPU-counter signals (`gpu_util_recovered`, `memory_pressure_low`) are inoperative is specific to saturated, high-concurrency serving and may not hold under lighter load, where utilization is more likely to dip below the recovery threshold. The RQ4 composite score uses a fixed weighting (40% accuracy, 25% premature-stop avoidance, 20% re-escalation avoidance, 15% cost-efficiency) that is a judgment call rather than an empirically derived tradeoff, and a different weighting could reorder policy rankings. Finally, repetition counts are modest (five microbenchmark repetitions, three vLLM seeds, eighteen

replay streams), which limits the statistical power of the reported confidence intervals.

V. CONCLUSION

This paper presented an adaptive GPU tracing controller that resolves the tension between profiling cost and diagnostic confidence in LLM serving systems by collecting cheap evidence continuously and activating short, bounded Nsight Systems bursts only when needed, stopping as soon as the kernel-level diagnosis has stabilized. Across NVIDIA L4 and A100 GPUs, six vLLM serving scenarios, and three microbenchmark workloads, the controller reduced profiler duration by 21.7%–48.5% and captured kernel instances by 42.9%–78.2% relative to manual fixed-window profiling, while preserving a diagnosis match rate of 1.0 and introducing less than 1% mean p95 latency overhead. Among the six stopping policies evaluated, Hybrid Stop and Counter-Recovery Stop eliminate re-escalation risk at modest additional cost, Stability Stop and Marginal Utility Stop offer a cheaper evidence-driven alternative, and a single Fixed Burst is unsafe under ambiguous first-window conditions; of the nine candidate stopping signals, only diagnosis stability reliably avoids firing on unconverged diagnoses. Together these results show that adaptive, evidence-driven control of heavy GPU profiling can substantially reduce tracing cost without sacrificing diagnostic accuracy, offering a practical alternative to today’s all-or-nothing approach to GPU profiling in production LLM serving.